



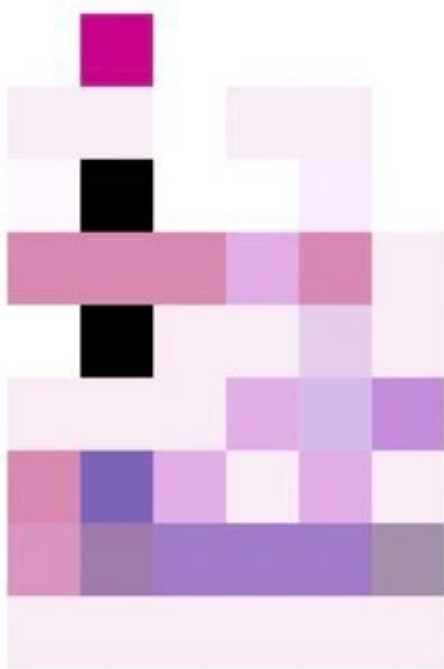
3>SHAKE

30分でわかる アーキテクチャ モダナイゼーション

書籍の全体像を掴む

2026/02/19 3-shake SRE Tech Talk 特別回

@nwiizo 30min (19:35~20:05)



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

**あらゆるサービスを
連携するハブになる**

クラウド型ETL/データパイプ
ラインサービスの決定版

Security



セキュリファイ

**事業者が抱える
セキュリティリスクを無くす**

ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



リランス

**良いエンジニアに
良い案件を**

フリーランスエンジニアに
「今よりいい条件」を



Sreakeのお仕事

クラウドネイティブなアプローチで、お客様の事業をより安全に、競争力のあるサービスへ

提供サービス

SRE/DevOps支援

- Kubernetes構築・運用
- クラウドネイティブ化推進
- Observability導入

アーキテクチャモダライゼーション

- 現状分析・戦略策定
- 段階的な移行支援
- 内製化・伴走支援

データ活用支援

- データ基盤構築
- BigQuery/Snowflake
- 分析基盤最適化

ご依頼・ご相談お待ちしております

<https://sreake.com/>



この発表で解決できること

こんな悩みを持っていませんか？

500ページの本を読む時間がない

→ 17章の構成と各章の核心メッセージを30分で掴める地図を提供します。全体像を先に知ること、必要な章だけ深掘りできるようになります

どこから読めばいいか不明

→ 読者のタイプ別に読書ルートを案内します。技術者、マネージャー、経営層—それぞれの関心に合った入口から読み始められます

経営層に説明できる言葉がない

→ 技術用語を使わずにモダナイゼーションの価値を語れるフレームワークを紹介します。「なぜ今やるべきか」を経営の言葉で伝える武器になります

今日の30分で「この本、読んでみよう」と思えるようになること

※30分で17章を扱う以上、各手法の実践詳細は書籍本体に委ねます。原則は組織規模やドメインを問わず適用できます



書籍の概要



アーキテクチャモダナイゼーション

- 著者: Nick Tune, Jean-Georges Perrin
- 翻訳: 株式会社スリーシェイク
- 原書: Manning Publications, 2024

本書の一言要約

レガシーシステムを競争優位性に転換するための包括的ガイド。技術だけでなく、ビジネス・組織・文化の変革を含む総合的なアプローチを提示。

本日の流れ

1. **Part 1: Why（理解）** — なぜモダナイゼーションが必要か
2. **Part 2: Discovery（発見）** — 現状をどう把握するか
3. **Part 3: Design（設計）** — 何を作るべきか

4. **Part 4: Organization（組織）** — 誰がどう作るか
5. **Part 5: Technical（技術）** — どう作るか
6. **Part 6: Execute（実行）** — どう進めるか

書籍の構成に沿って、各パートの核心を紹介します



書籍は6つのパートで構成されている

Part	テーマ	章	問い	主要な概念・手法
1. Why	理解	Ch.1-3	なぜモダナイゼーションが必要か？	BVSSH、独立した価値ストリーム、負のサイクル
2. Discovery	発見	Ch.4-6	現状をどう把握するか？	リスニングツアー、Wardley Mapping、プロダクトタクソノミー
3. Design	設計	Ch.7-9	何を作るべきか？	イベントストーリーミング、Bounded Context、コンテキストマッピング
4. Organization	組織	Ch.10-11	誰がどう作るか？	コアドメインチャート、Team Topologies、認知的負荷
5. Technical	技術	Ch.12-14	どう作るか？	疎結合アーキテクチャ、IDP、Data Mesh
6. Execute	実行	Ch.15-17	どう進めるか？	AMET、アダプティブロードマップ、学習文化

Why → Discovery → Design → Organization → Technical → Execute の流れ



Part 1: Why (理解)

Ch.1-3

最初の問い：「なぜモダナイゼーションが必要なのか？」

Ch.1 アーキテクチャモダナイゼーションとは

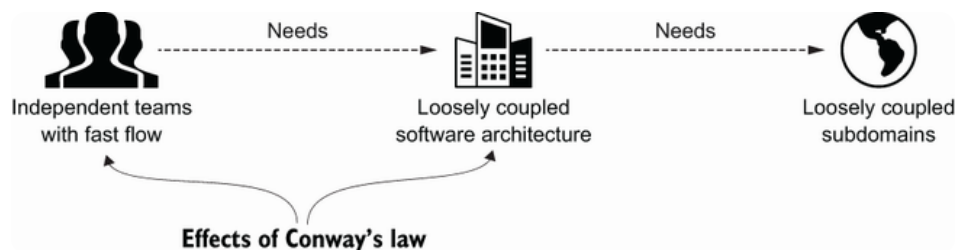


Figure 2.1 Fast flow requires loosely coupled software architecture, which requires loosely coupled domains. より引用

定義：時代遅れのアーキテクチャをビジネス競争優位性へ転換すること

単なる技術のアップグレードではない。ビジネス・組織・技術を同時に変革する**ソシオテクニカル**なアプローチ。書籍はこの「同時に変革する」という視点を一貫して貫いている。

核心的な概念

- **BVSSH** — Better Value Sooner Safer Happier（品質・価値・速度・安全・幸福の5軸）
- **独立した価値ストリーム** — ドメイン整合、チーム自律、成果志向、疎結合

3-6ヶ月以内に価値を提供し始めることができる長期投資

ソシオテクニカルとは何か

ソシオテクニカル = 社会的システム（人・組織・文化）と技術的システム（アーキテクチャ・ツール）を**一体として**設計すること

技術だけ変えても失敗する例

やったこと	結果	根本原因
マイクロサービス化	チーム間調整が増えて逆に遅くなった	チーム構造を変えずにアーキテクチャだけ変更
クラウド移行	同じ非効率なプロセスがクラウドで動くだけ	承認フローや組織文化はそのまま
最新フレームワーク導入	使いこなせる人がいなくて品質低下	スキルアップの時間と予算を確保せず

ソシオテクニカルに変えると

技術の変更	組織の変更	文化の変更
マイクロサービス化	チームを機能別から価値ストリーム別へ	「自分たちで決める」文化へ
クラウド移行	運用チームをプラットフォームチームへ	実験と学習を奨励
最新フレームワーク	学習時間を業務時間に組み込む	失敗から学ぶ文化へ

技術だけ変えても失敗する。「マイクロサービス化したのに遅くなった」は、ほぼこのパターン。



BVSSHで成功を測る

なぜBVSSH？ — Jon Smartが提唱したフレームワーク。「速度」だけを追うと品質が下がり、「品質」だけを追うと遅くなる。**5つ同時に改善**することで、本当の成功が測れる。

軸	問い	目指す姿	測定指標例
Better	品質は上がったか？	テスト自動化で品質担保	バグ密度、顧客満足度
Value	顧客に価値を届けたか？	顧客の課題を解決	機能利用率、コンバージョン
Sooner	早く届けられたか？	2週間でデプロイ	リードタイム、デプロイ頻度
Safer	リスクは下がったか？	障害時も即座に復旧	MTTR、変更失敗率
Happier	チームは幸せか？	成長実感、やりがい	eNPS、離職率

「速度 vs 品質」はトレードオフではない。両方追求できる。



独立した価値ストリームとは

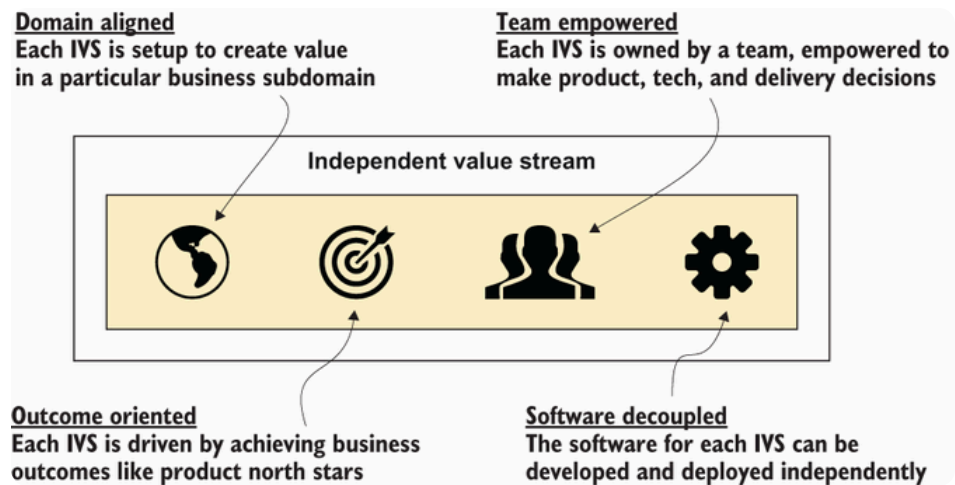


Figure 1.5 The four key characteristics of an independent value stream より引用

4つの特徴

ドメイン整合 —
ビジネスドメイン
と一致した境界

チーム自律 — 他
チームへの依存な
く意思決定

成果志向 — 機能
ではなくビジネ
ス成果で評価

疎結合 — 他スト
リームと最小限の
依存関係

独立した価値ストリームの数 = 組織の並列処理能力

目指す姿はわかった。では、どう始めるか？

Ch.1ではソシオテクニカルなアプローチと独立した価値ストリームという目指す姿を見てきました。ゴールは明確になった。

しかし「良い目標」があるだけでは始められない。始める前に踏んではいけない地雷がある。経営層を巻き込む方法を知らなければ予算も人も集まらない。Ch.2-3では、**旅に出る前の準備**——避けるべき罠、経営層への説明、ビジネスケースの作り方を扱います。

Ch.2 旅への準備

技術的な準備と同じくらい、組織と人的準備が重要。レガシーシステムには「負のサイクル」が潜んでいる――技術的負債の蓄積 → 開発速度の低下 → 優秀な人材の流出 → さらなる負債の蓄積。この悪循環を断ち切るための準備がCh.2のテーマ。

準備すべきこと

経営層の準備

本気のコミットメント、長期的視点。「3ヶ月で成果を見せつつ、3年かけて変える」覚悟。信頼は一朝一夕では生まれない――コミットメントの履行を積み重ねることではか築けない。

避けるべき罠

銀の弾丸思考――ツールやフレームワークを入れれば解決すると信じること。**構造とプロセスの幻想**――組織図を変えれば文化が変わると思ひ込むこと。**ボルトオン型**――既存の体制に変革チームを後付けするだけで終わること。次のスライドで詳しく解説。

モダナイゼーションは技術プロジェクトではなく、人と組織の変革である



負のサイクルとは

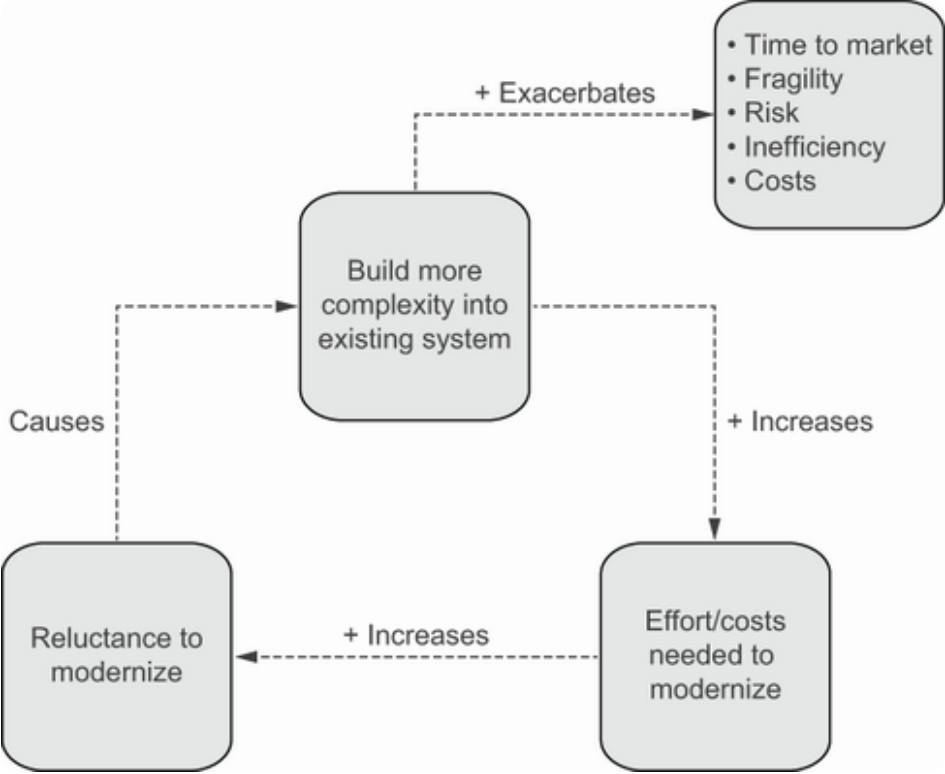


Figure 1.1 The negative cycle of declining architecture health より引用

負のサイクル = レガシーシステムが組織を蝕み、組織がさらにレガシーを生む悪循環

具体的な症状

段階	何が起きるか	結果
1	「時間がない」でリファクタリング先送り	技術的負債が蓄積
2	変更にかかる時間が増える	リリース頻度低下
3	優秀なエンジニアが離職	さらに開発速度低下
4	障害対応が主業務に	イノベーション停止
5	競合に市場を奪われる	売上減少 → 投資減少

「時間がない」は原因ではなく症状。負のサイクルの中にいると、投資しない理由が自動生成される

避けるべき3つの罠

1. 銀の弾丸思考

「マイクロサービスにすれば解決」「クラウドに移行すれば解決」

問題: 技術だけで組織の問題は解決しない

対策: 技術は手段、目的はビジネス成果

2. 構造とプロセスの幻想

「組織図を変えれば変わる」「新しいプロセスを導入すれば変わる」

問題: 文化とマインドセットが変わらなければ形骸化

対策: 小さな成功体験から文化を変える

3. ボルトオン型

古いシステムの横に新しいシステムを「付け足す」

問題: 複雑性が倍増、依存関係も増加

対策: 段階的な置き換え、Strangler Fig パターン

厄介なのは、罠にはまっている側は「正しいことをしている」と思っていること

「マイクロサービスにすれば解決する」のか？

この反論はほぼ100%出ます。「銀の弾丸思考」の最も典型的な例なので、ここで正面から扱います。

マイクロサービスが解決すること・しないこと

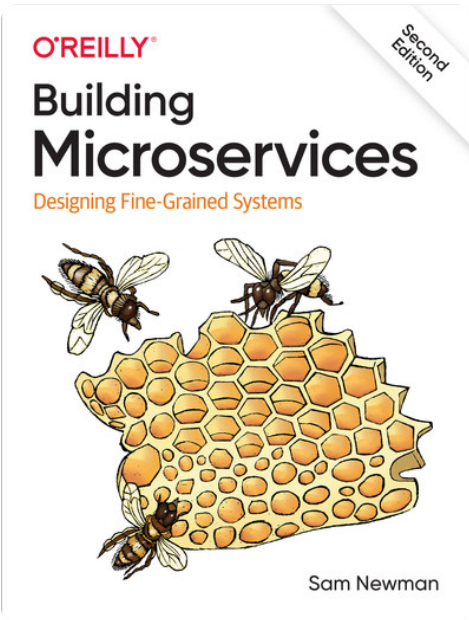
解決すること	解決しないこと
デプロイの独立性	チーム間の責任の曖昧さ
技術選択の自由度	ドメイン境界が間違っている問題
障害の局所化	組織文化・承認フローの遅さ

マイクロサービスは**アーキテクチャパターンの1つ**であって、モダナイゼーションの目的ではない。ドメイン境界が間違ったまま分割すると、モノリスより悪い「分散モノリス」ができる。Ch.9でドメイン境界を正しく引く方法を扱うのは、この罠を避けるため。

技術選定の前にドメイン戦略。順番を間違えると、新しい負債が生まれる。



Building Microservicesも同じ結論に至っている



Sam Newman, O'Reilly, 2021

Sam Newmanの **Building Microservices, 2nd Edition** はマイクロサービスの実践ガイドとして最も広く読まれている書籍。その著者が第2版で強調したのは、**独立デプロイ可能性 (Independent Deployability)**。こそがマイクロサービスの本質的な価値であり、マイクロサービスという形態自体はゴールではないということ。

書籍の3部構成と核心メッセージ

Part	テーマ	Newmanの主張
I. Foundation (Ch.1-4)	境界の設計	情報隠蔽・凝集性・DDDで境界を引く。境界が間違っていれば分割しても意味がない
II. Implementation (Ch.5-13)	実装と運用	通信・テスト・デプロイ・セキュリティ。運用コストに見合う価値があるか先に問え
III. People (Ch.14-16)	組織と人	Conway's Law、進化的アーキテクト。最大のスケーリングコストは調整コスト

Newman: 「マイクロサービスはほとんどのスタートアップに向かない」。問うべきは「独立デプロイ可能性が必要か」。

エンジニアだけが「やりたい」では始まらない

罨を知っているだけでは十分じゃない。銀の弾丸を避け、ボルトオン型を避け、正しいドメイン境界で分割するー技術的には正解でも、**経営層が「やるべきだ」と判断しなければ**、モダナイゼーションには予算もチームもつかない。

現場のエンジニアが「技術的負債がひどい」と訴えても、経営層には「で、売上にどう影響するの?」と返される。この翻訳ができないまま始めたプロジェクトは、最初の予算削減で真っ先に切られる。Ch.3では、技術者の言葉をビジネスの言葉に翻訳する方法を扱います。

Ch.3 ビジネス目標

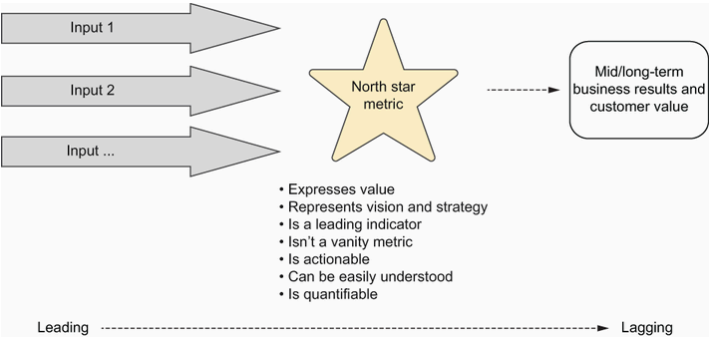


Figure 3.2 より引用

「技術的負債を返済したい」だけでは経営層は動かない。ビジネス言語で語る必要がある。

経営層が気にしているのは売上・コスト・リスク・競争優位性。「アーキテクチャを刷新したい」ではなく「リリース頻度を上げて市場投入を早める」「障害対応コストを減らして人員を新規開発に回す」と言い換える。モダナイゼーションの提案は、これらの経営指標にどう効くかを数字で示さなければ予算はつかない。

経営層が理解できるビジネス目標の例

技術者の言葉	ビジネスの言葉	経営層へのアピールポイント
マイクロサービス化したい	新機能のリリースを月1回から週1回に	競合より先に市場投入、顧客フィードバック加速
技術的負債を返済したい	障害対応コストを年間30%削減	運用コスト削減、エンジニアを新規開発に再配置
モダンな技術を使いたい	採用競争力を上げて優秀な人材を確保	人材獲得競争で優位、離職率低下
テスト自動化したい	本番障害を50%減らし顧客満足度向上	顧客離反防止、ブランド価値向上

経営層への説明テンプレート

「[ビジネス課題] を解決するために、[技術的アプローチ] を行い、[期間] で [測定可能な成果] を達成します」



技術の言葉ではなく、ビジネスの言葉で語る

Ch.4 リスニング&マッピングツアー

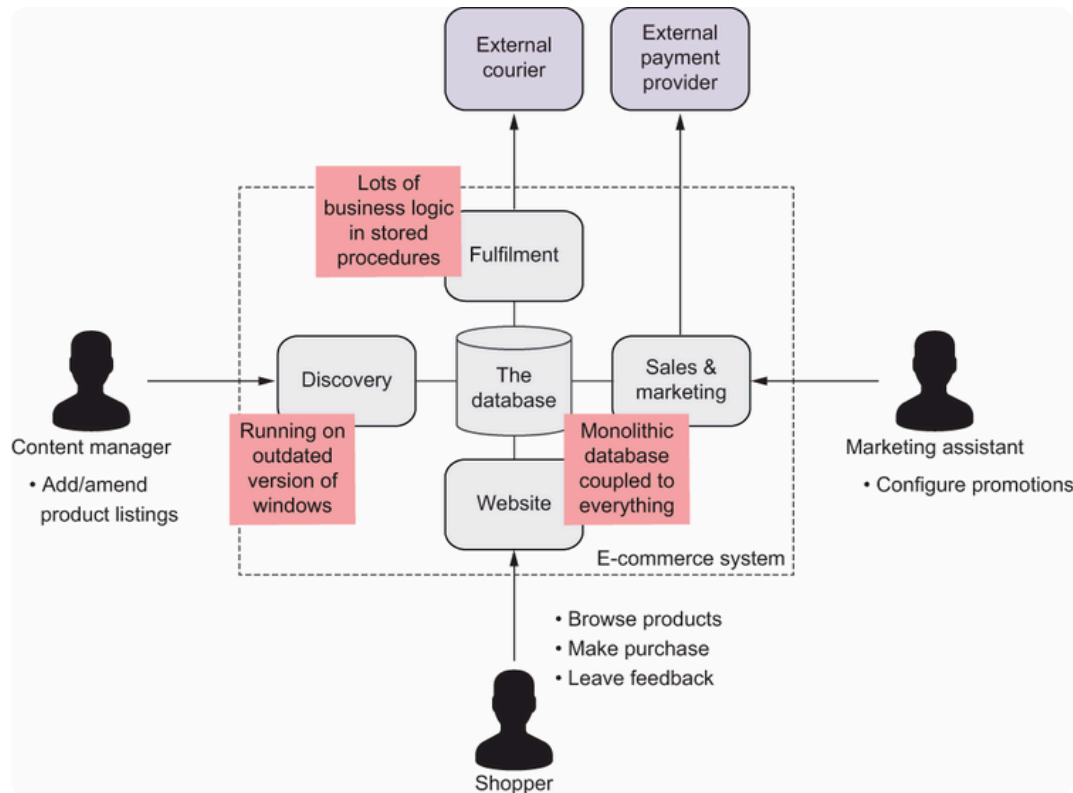


Figure 4.4 A visual summary from a listening tour より引用

ビジョンを押し付けるのではなく、現場の課題を理解することが先決

実施内容

リスニングツアー — 経営層からエンジニアまで、多様なステークホルダーに1対1で話を聴く。何に困っているか、何を変えたいか、何が怖いのか。答えを持っていく場ではなく、問いを持っていく場。現場の言葉をそのまま記録することが重要。

マッピングツアー — システム間の依存関係、チーム構造、責任範囲を図に起こして可視化する。誰も全体像を把握していないことが多く、描いてみて初めて「ここが繋がっていたのか」という発見が生まれる。

語るのではなく、聴くことが説得するための最初のステップ

リスニングツアーの実践方法

目的: 現場の痛みと願望を、先入観なく理解する。自分の仮説を確認しに行くのではなく、相手の世界を知りに行く。「こうすべき」ではなく「何が起きているか」を聴く姿勢が重要。

聴くべき質問

痛みを探る

- 日常業務で最も困っていることは？
- 「もっとこうだったら」と思うことは？
- 顧客への価値提供を妨げているものは？

同じ質問でも、立場が違えば答えが変わる。その差分こそが組織の本当の課題を映し出す。

理想を探る

- 半年後、どうなっていたら成功か？
- チームが最も活躍できる条件は？
- 会社のどこに誇りを感じるか？

痛みだけでなく、誇りや理想も聴く。人は「何を直したいか」だけでなく「何を守りたいか」にも本音が出る。

インタビュー対象

経営層 → プロダクトマネージャー → 開発者 → 運用チーム → カスタマーサポート。階層を横断して聴くことで、同じ事象に対する認識のズレが浮かび上がる。

聴いた内容をそのまま伝えることで、共感の連鎖が生まれる



マッピングツアーで現状を「地図」にする

マッピングツアー = システム・チーム・依存関係を可視化して、現状を「地図」にする

システム地図と依存関係地図

システム地図

- どんなシステムがあるか
- どのシステムが重要か
- どれがレガシーか

依存関係地図

- 誰が誰に依存しているか
- ボトルネックはどこか
- 変更の影響範囲は

まずシステムの全体像を把握し、次にシステム間の依存関係を明らかにする。**何があるか**と**どう繋がっているか**が最初の問い。



マッピングツアーでチームと痛みを可視化する

チーム地図と痛みの地図

チーム地図

- 誰がどのシステムを担当
- チーム間の境界は明確か
- 責任の空白地帯はないか

痛みの地図

- どこで問題が起きているか
- 頻繁に障害が起きる箇所
- 変更が困難な箇所

成果物の例

「決済システムは在庫・配送・顧客管理の3システムに依存。変更のたびに4チームの調整が必要。」

地図がなければ、どこに行くべきか分からない

現場の声が集まった。次は投資判断

リスニング&マッピングツアーで「痛み」と「構造」が見えてきました。現場の声は集まった。しかし、痛い場所をすべて同時に直す余裕はどの組織にもない。限られた予算と人員を**どこに集中投資すべきか？** その判断を感覚ではなく構造的に可視化するのが、Ch.5のWardley Mappingです。

Ch.5 Wardley Mapping

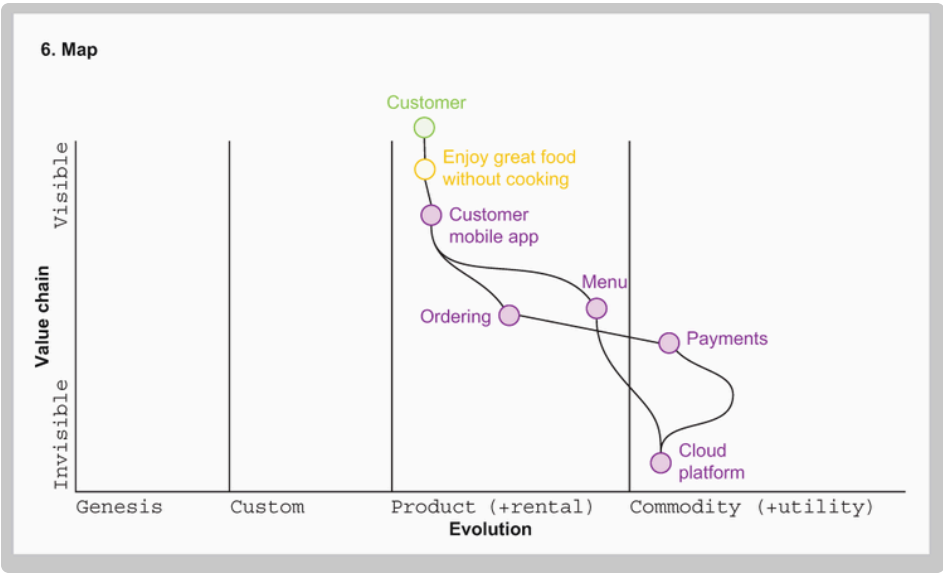


Figure 5.7 Step 6 of the Wardley Mapping Canvas, a Wardley Map より引用

バリューチェーンと技術進化の関係を可視化して戦略を立案する

Simon Wardleyが考案したマッピング手法。地図なしに戦争はできないのと同じで、自社の技術ランドスケープを地図にしなければ戦略は立てられない。「何を内製し、何を買い、何を捨てるか」を構造的に議論するためのツール。

Wardley Mapの軸

軸	内容
縦軸	ユーザーニーズからの距離（価値チェーン）
横軸	進化段階（Genesis → Custom → Product → Commodity）

戦略サイクル

Purpose（目的）→ Landscape（景観）→ Climate（気候）→ Doctrine（教義）→ Leadership。まず目的を定め、現在地を地図にし、変化の方向を読み、原則に従い、最後にリーダーシップで決断する。

戦略を視覚化することで、組織全体が同じ方向を向く



Wardley Mapで戦略を可視化する

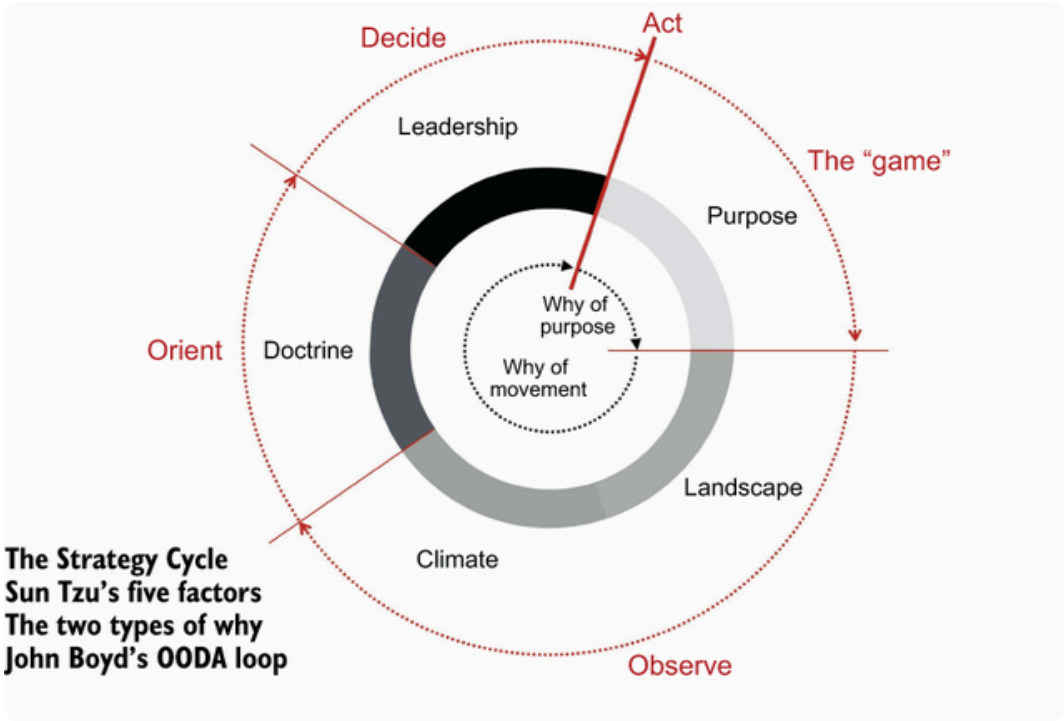


Figure 5.1 The Strategy Cycle (Source: Simon Wardley) より引用

進化段階ごとの戦略

段階	特徴	戦略	例
Genesis	不確実	実験、失敗を許容	新しいAI機能
Custom	差別化要因	自社で作り込む	独自のレコメンド
Product	複数選択肢あり	最適なものを選ぶ	CRMツール
Commodity	標準化済み	購入、クラウド利用	メール、ストレージ

よくある間違い

間違い	正しいアプローチ
Commodity（認証）を自社開発	Auth0, Okta等のIDaaS活用
Genesis（新機能）を外部委託	内製で知識とノウハウを蓄積

「どこで勝負するか」を明確にして、メリハリをつける



戦略はできた。次は共通言語を揃える

Wardley Mapで「どこに投資するか」が見えた。CoreドメインとGenericドメインの区別もついた。しかし、経営層は「プロダクト」、PdMは「機能」、エンジニアは「サービス」と呼んでいて、**同じ粒度で議論する言語がない**と会話がかみ合わない。Ch.6のプロダクトタクソミーは、全員が同じ地図を見て話すための共通語彙です。



Ch.6 プロダクトタクソミー

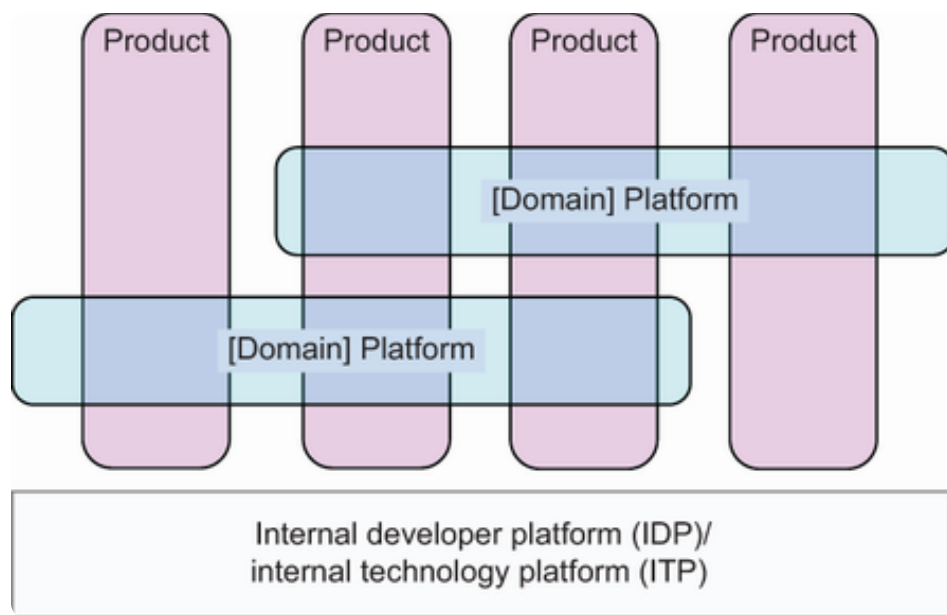


Figure 6.6 A product taxonomy より引用

アーキテクチャを記述するための統一された構成要素の定義

階層構造

独立した価値ストリーム
└─ ドメイン
 └─ サブドメイン
 └─ プロダクト
 └─ 機能 (Feature)

マクロ視点からミクロ視点まで一貫した言語で語れるようになる。

アーキテクチャと組織の橋渡し

Ch.7 ビッグピクチャーイベントストーリーミング

なぜやるのか？ — エンジニアだけで設計すると、ビジネスの現実と乖離する。ビジネス担当者だけで語ると、技術的制約が見えない。**全員で一緒に**ビジネスを可視化することで、共通理解が生まれる。

具体的な進め方

ステップ	やること	所要時間目安
1. 嵐を起こす	オレンジの付箋にイベントを書きまくる。順番は気にしない	30分
2. タイムラインを作る	イベントを時系列に並べる	30分
3. 問題を見つける	赤い付箋で「ここがおかしい」「ここが分からない」を貼る	20分
4. 境界を引く	自然なまとまりを見つけて線を引く	20分

成果物

ビジネスの全体像を可視化した壁（通常8m～15mの長さになる）

「混沌」から始めて「構造」を発見する



イベントストリーミングの様子



Figure 7.1 Big Picture EventStorming より引用

付箋の色

色	要素
オレンジ	ドメインイベント（～された）
青	コマンド（～する）
黄色	アクター（誰が）
紫	ポリシー（自動処理）
赤/ピンク	ホットスポット（問題点）

フロー

Phase 1: 発散 → **Phase 2:** 時系列整理 → **Phase 3:** ホットスポット特定 → **Phase 4:** 境界線

ホットスポットは宝の山

ホットスポットが多い場所こそ、モダナイゼーションの優先候補になる。

ホットスポットが集中する場所	意味
部門の境界	チーム間の連携に問題
特定のプロセス	ボトルネック、複雑すぎる
特定のシステム	レガシー、技術的負債

集中している箇所を俯瞰すると、組織の構造的な問題が浮かび上がる。個別のホットスポットではなく、**パターン**を読み取ることが重要。

ホットスポット = モダナイゼーションの優先候補



Ch.8 プロダクトとドメインのモダナイゼーション

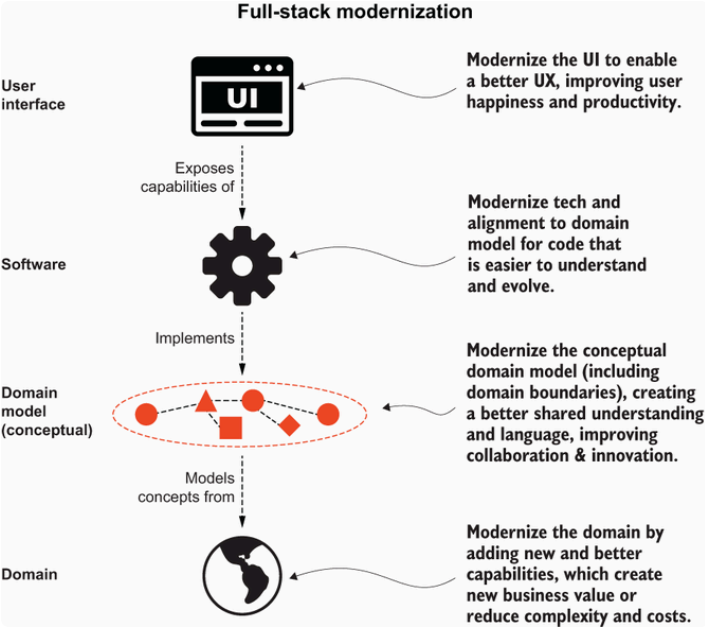


Figure 8.1 より引用

問題: 技術を新しくしても、業務の複雑さや使いにくいUIをそのまま引き継ぐと、新しくて使いにくいシステムができるだけ。

フルスタックモダナイゼーション

レイヤー	古いシステムの問題例	モダナイゼーションで改善
UX	10画面遷移で注文完了	3クリックで注文完了
技術	オンプレ・モノリス	クラウドネイティブ・マイクロサービス
ドメインモデル	「商品」に100以上の属性	文脈ごとに必要な属性だけ
ビジネスプロセス	5部署の承認が必要	自動化+1承認

技術だけでなく、業務・UX・ドメインも同時に再設計する



Strangler Figパターン

Strangler Fig = 熱帯の絞め殺しの木に由来する名前。Martin Fowlerが2004年のブログ記事で紹介。レガシーシステムを段階的に新システムに置き換えるパターンで、Big Bang Rewriteのリスクを回避する。仕組みはシンプルで、ユーザーとレガシーの間にルーターを置き、新機能から順に新システムへ振り分ける。

段階的な移行

フェーズ	やること	リスク
1	ルーター導入、トラフィック分岐	低
2	1機能を新システムに移行	低
3	成功を確認、次の機能を移行	低
4	全機能移行完了、レガシー廃止	低

Big Bang Rewriteとの比較

	Big Bang	Strangler Fig
リスク	高（全機能同時）	低（1機能ずつ）
価値提供	完成まで0	段階的に提供
ロールバック	困難	機能単位で可能

少しずつ絞め殺す。一気に殺さない。



移行戦略は決まった。次は「どこで切るか」

Strangler Figパターンで段階的に移行できることがわかりました。しかし「段階的に」と言うのは簡単でも、**どこで区切るか**を間違えると、分割した先で新たな密結合が生まれる。切り方の基準がなければ、結局は声の大きい人の勘で決まってしまう。Ch.9ではDDDのBounded Contextを使って、ドメインの構造から境界線を見つける方法を扱います。

Ch.9 ドメインとサブドメインの識別

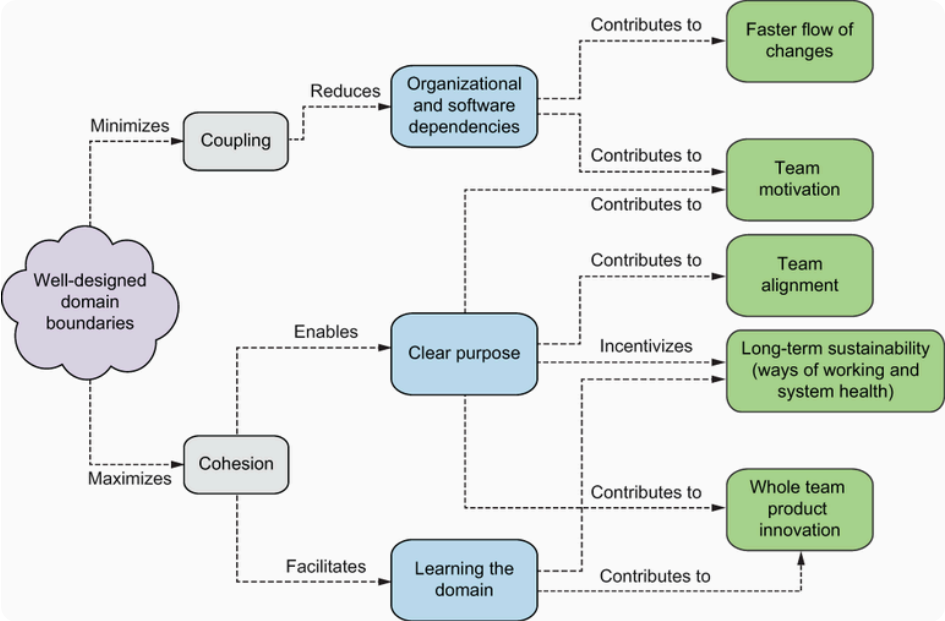


Figure 9.2 Bounded Context and its relationships より引用

よいドメイン境界は、依存関係を減らし、チームの自律性を高める。DDDの Bounded Contextは「同じ言葉が同じ意味を持つ範囲」を定義する概念で、この範囲がそのままサービスやチームの境界になる。境界を間違えると、分割したはずのサービス同士が密結合し、変更のたびに複数チームの調整が必要になる。

ドメイン境界を見つけるヒューリスティック

変更頻度

一緒に変わる概念をまとめる。変更理由が同じものは同じ境界に。

凝集性

関連するビジネス概念をグループ化。「注文」と「配送」は近いが別の関心事。

チーム境界

1チームの認知負荷に収まる範囲に切る。大きすぎると把握できない。

科学ではなく、アート＋経験則。複数の視点を組み合わせて判断する



ドメイン境界を見つける9つのヒューリスティック

1. ユビキタス言語の境界

同じ言葉が異なる意味を持つ場所で区切る

2. ビジネスケイパビリティ

ビジネスが「できること」単位で分ける

3. 揮発性（変更頻度）

変更頻度が異なるものを分離する

4. 規制とコンプライアンス

法規制の対象となる領域を分離

5. サブドメインの分離

Core / Supporting / Generic で分類

6. サービス配置プレースメント

デプロイ単位との整合性

7. 組織構造

Conway's Law — 組織とアーキテクチャは相似形

8. 既存の境界を尊重

レガシーシステムの境界を活用

9. イベントストリーミングの結果

ワークショップで見た自然な区切り

複数のヒューリスティックを組み合わせる総合的に判断する



コンテキストマッピング

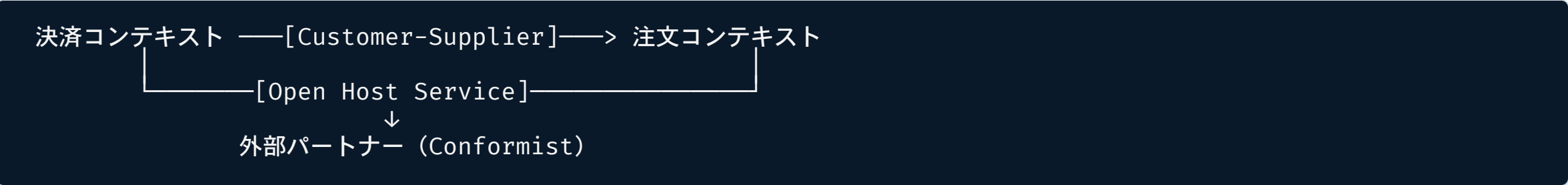
コンテキストマッピング = Eric EvansのDDDで定義された、Bounded Context間の関係性を可視化する手法。誰が誰に依存し、その力関係はどうなっているかを明示する。

実務でよく使う3パターン

パターン	関係性	例
Customer-Supplier	上流-下流の依存	決済チーム(上流)→注文チーム(下流)
Anti-Corruption Layer	翻訳層を設ける	レガシーとの間に変換層
Open Host Service	公開API	誰でも使えるAPIを提供

他にもPartnership、Shared Kernel、Conformistなどのパターンがある。書籍Ch.9で詳述。

コンテキストマップの例



Ch.10 戦略的ITポートフォリオ

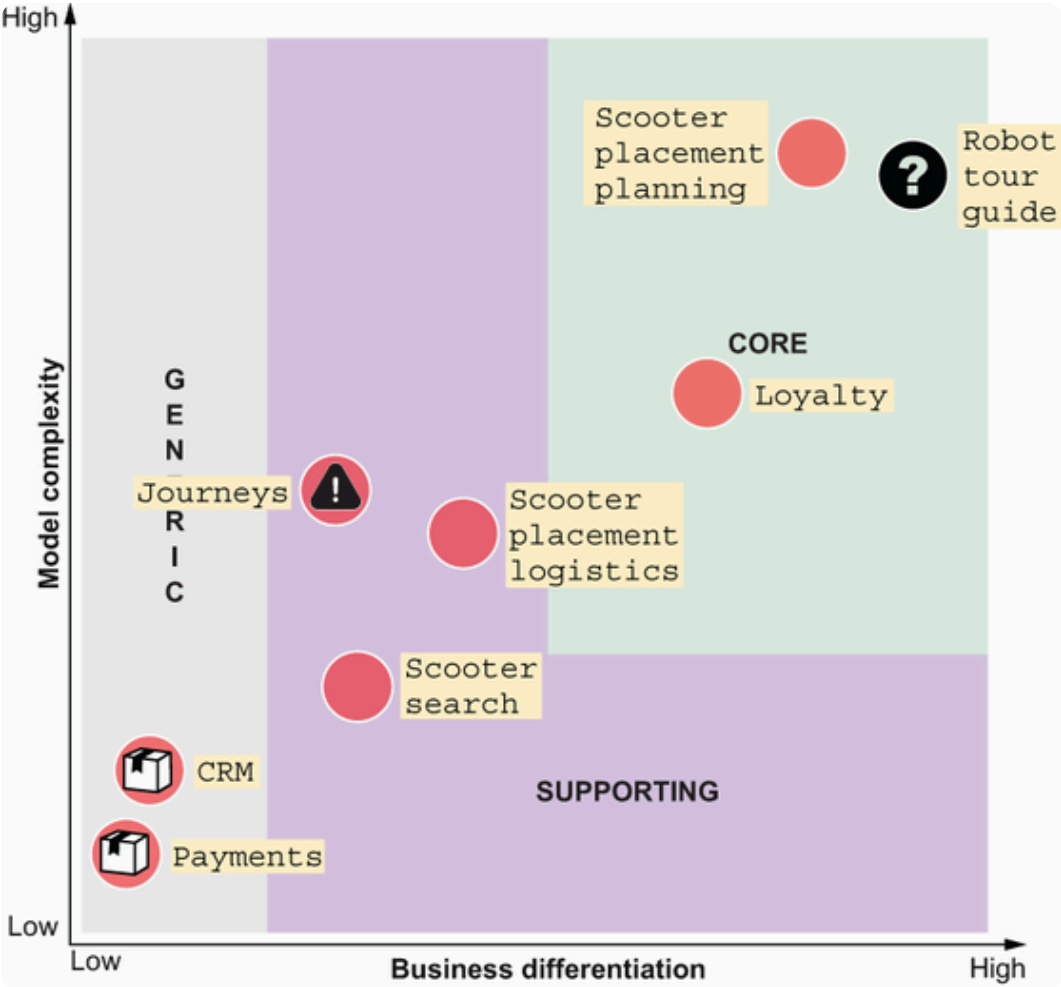


Figure 10.5 The Core Domain Chart より引用

すべてのドメインに等しく投資するのではなく、コア・支援・汎用に分類して投資にメリハリをつける。コアドメインチャートは、Wardley Map で見た進化段階をチーム投資に落とし込むツール。

ECサイトの例

ドメイン	分類	投資戦略	理由
レコメンド	コア	自社開発・AI投資	競合との差別化ポイント
在庫管理	支援	バランス型	重要だが差別化要因ではない
認証	汎用	SaaS購入	自社で作る意味がない

間違った投資の例

- ✗ 認証を自社開発 → セキュリティリスク増、本業から逸脱
- ✗ レコメンドを外部委託 → 差別化要因を手放す

「どこで勝負するか」を明確にして投資を集中する



投資優先順位は2x2マトリクスで決める

2x2マトリクス = ビジネス価値と技術的負債の2軸で、投資すべき対象を判断する

ビジネス価値 × 技術的負債

	技術的負債：低	技術的負債：高
ビジネス価値：高	● 維持・拡張	● 最優先で改善
ビジネス価値：低	● 現状維持	● 廃止を検討

右上の「高価値・高負債」がモダナイゼーションの最優先ターゲット。ビジネスに重要なのに問題だらけの領域。逆に左下の「低価値・低負債」は放置でいい。

✗ 「技術的に面白い」で優先度を決めるのはNG → ビジネス価値と技術的負債の両方で判断



優先順位の考え方と判断基準

優先度の順序

優先度	対象	理由
1	高価値・高負債	ビジネスに重要なのに問題だらけ
2	高価値・低負債	競争優位を維持・強化
3	低価値・高負債	廃止してリソースを解放

低価値・低負債は優先度リストに入らない。触る理由がないものに時間を使わない。

ただし「低価値」は現時点の評価であり、将来の戦略的価値は別途判断が必要。ポートフォリオ全体を見て、限られたリソースをどこに集中させるかが本質的な問い。

「全部やる」は「何もやらない」と同じ

Ch.11 Team Topologies

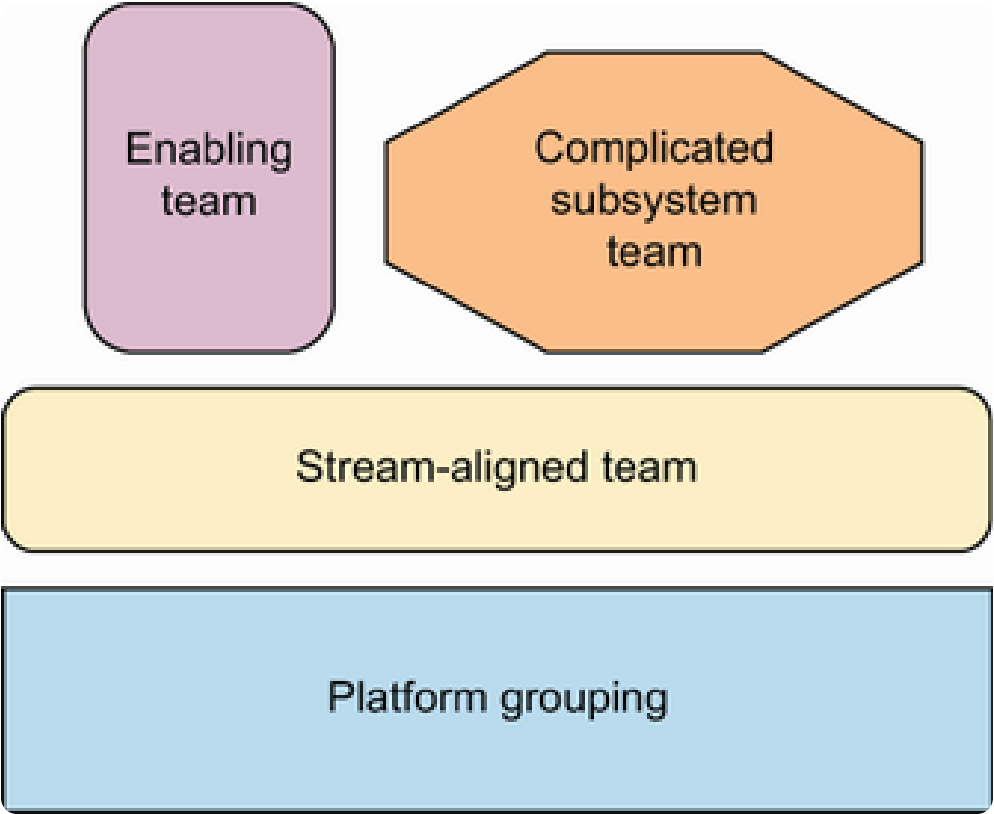


Figure 11.3 The four team types in Team Topologies より引用

Matthew Skelton & Manuel Paisによる組織設計のフレームワーク。チームを4つのタイプに分類し、チーム間の関わり方を意図的に設計する。

4つのチームタイプ

チーム	何をする？	例
Stream-aligned	顧客価値を直接届ける	決済チーム、検索チーム
Platform	他チームの仕事を楽にする	インフラチーム、CI/CDチーム
Enabling	新しい技術・手法を教える	SREコーチ、アジャイルコーチ
Complicated-subsystem	専門知識が必要な部分を担当	ML基盤、暗号化エンジン

大半のチームはStream-alignedであるべきで、他の3タイプはStream-alignedチームのフローを加速するために存在する。

Conway's Law: 組織構造 = システム構造。これは選択ではなく物理法則。



「組織を変えずにアーキテクチャだけ変えたい」

よく聞く反論です。「組織変更は政治的に難しい。せめて技術だけでも先に変えたい」——気持ちわかります。しかし Conway's Lawは「法則」であって「提案」ではない。

技術だけ変えた場合に何が起きるか

やったこと	期待した結果	実際の結果
マイクロサービス分割	チームの自律性向上	同じ承認フローが残り、調整コスト増
API化	疎結合	組織の権限構造がAPIの設計を歪める
クラウド移行	デプロイ速度向上	変更管理委員会がボトルネックのまま

組織を変えずに技術だけ変えると、**新しいアーキテクチャが古い組織構造の形に引き戻される**。逆コンウェイ戦略——望ましいアーキテクチャに合わせて組織を設計する——が書籍の一貫した主張。

組織を変える覚悟がないなら、アーキテクチャも変わらない。



認知的負荷がチーム設計を制約する

問題: 優秀なエンジニアでも、担当範囲が広すぎると「全部を頭に入れておく」ことができなくなる。これが認知的負荷。

認知的負荷が高いとどうなる？

症状	結果
「あのシステム、誰か分かる人いる？」	属人化、バス係数1
「変更の影響範囲が分からない」	恐怖でリリースできない
「新メンバーが戦力化するのに半年」	オンボーディングコスト増大
「障害対応で深夜に叩き起こされる」	燃え尽き、離職

対策

チームの担当範囲を「頭に入る」サイズに保つ — 具体的には5～9人のチームが「完全に理解できる」範囲のシステムを担当する。それ以上になったら分割。チームを大きくするのではなく、担当範囲を小さくする。

「理解できない」ものは「運用できない」。認知的負荷の限界が、チームの境界を決める。



チーム間の3つのインタラクションモード

インタラクションモード = チーム間でどう協力するかの3つのパターン。チームの種類だけでなく、チーム間の関わり方を意図的に設計することがTeam Topologiesの核心。

モード	説明	いつ使う	例
Collaboration	密に協力	新しい境界を探索中	新サービス立ち上げ時
X-as-a-Service	APIで疎結合	境界が明確	プラットフォーム利用
Facilitating	支援・コーチング	能力移転	イネーブリングチーム

モードは固定ではない。立ち上げ時は**Collaboration**で密に連携し、境界が明確になったら**X-as-a-Service**へ移行して疎結合にする。この進化を意図的に計画することが重要で、いつまでもCollaborationのままだと認知負荷が膨らみ続ける。



Ch.12 疎結合なソフトウェアアーキテクチャ

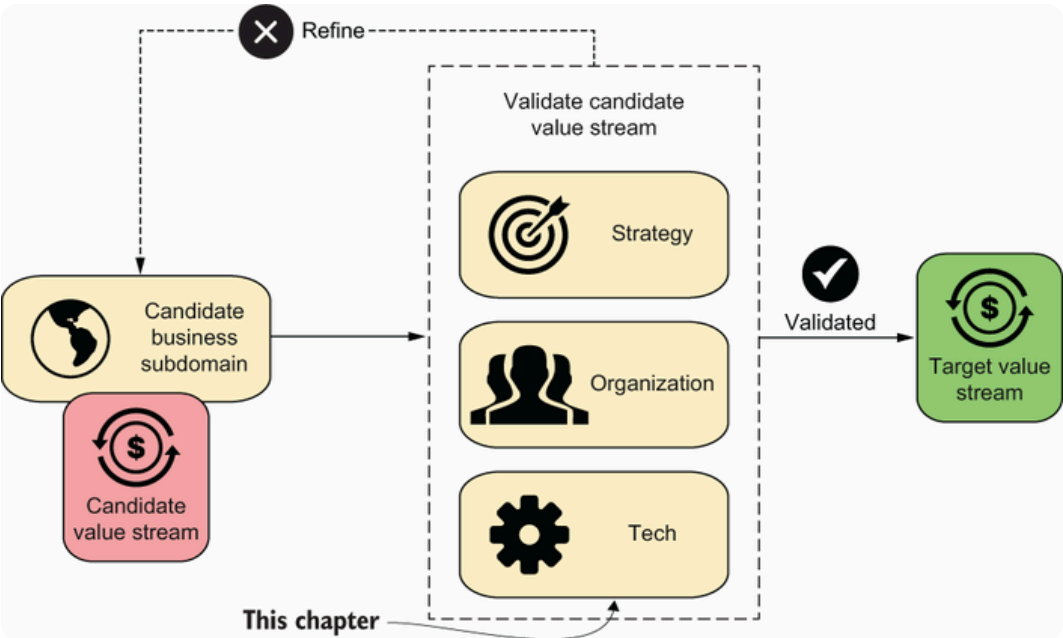


Figure 12.1 Loosely coupled software architecture より引用

なぜ疎結合が重要か？

密結合だと: Aを変更するとBも変更が必要 → Bを変更するとCも変更が必要 → 調整会議が増える → リリースが遅くなる。チームの自律性を確保するには、技術的にも疎結合でなければ意味がない。

結合の4つのタイプ

タイプ	何を知っている？	例
契約的	APIの形式だけ	REST API
モデル的	ドメインの概念	「注文」の意味
機能的	ビジネスルール	割引の計算方法
侵入的	実装の詳細	DBのテーブル構造

上から下へ行くほど結合度が高く、変更の影響範囲が広がる。侵入的結合は「相手のDBを直接読む」ような状態で、最も避けるべきパターン。

契約的結合を目指す。知りすぎは密結合。



密結合から疎結合への移行

密結合の兆候 = 「1箇所変えたら10箇所壊れた」「リリースのたびに調整会議」

密結合の具体例と解決策

密結合パターン	問題	解決策
共有DB	1テーブル変更で全システム影響	API経由でアクセス
同期呼び出し	1サービス落ちると連鎖障害	イベント駆動に変更
共有ライブラリ	バージョン合わせが地獄	契約テストで独立
ドメイン知識の漏洩	他システムがビジネスロジック実装	ドメインサービスに集約

疎結合への道

【Before: 密結合】

注文 —[DB参照]—→ 在庫テーブル
 —[同期呼出]—→ 決済サービス

【After: 疎結合】

注文 —[イベント発行]—→ メッセージキュー
 ↓
 在庫サービス / 決済サービス

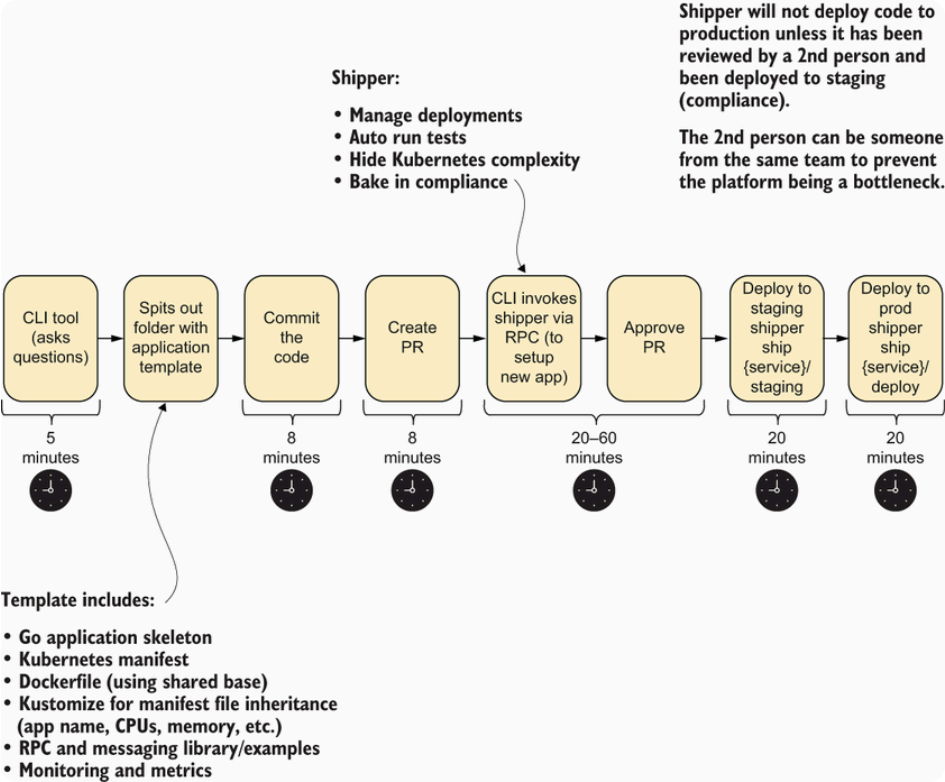


設計はできた。実践するための基盤は？

疎結合なアーキテクチャの設計方針が決まりました。しかし、新サービスを立ち上げるたびにインフラ構築で2週間、CI/CDパイプラインの整備で1週間――これでは設計は絵に描いた餅です。チームの自律性は、技術基盤が支えなければ実現しない。Ch.13のInternal Developer Platformは、**チームが自律的に動くための「舗装された道」**を提供します。



Ch.13 Internal Developer Platform



IDPとは、開発チームがインフラや環境構築をセルフサービスで行えるようにする社内プラットフォーム。Platformチームが構築・運用し、Stream-alignedチームの認知負荷を下げる。

IDPがない場合の問題

問題	結果
新規サービス立ち上げに2週間	「とりあえず既存サービスに追加」
環境構築の手順書が100ページ	新人が戦力化しない
デプロイのたびにインフラチームに依頼	ボトルネック化

IDPがある場合

Before	After
2週間かかる新規サービス立ち上げ	「Create New Service」ボタンで10分
100ページの手順書	テンプレートから自動生成
インフラチームに依頼	セルフサービスでデプロイ

「正しいやり方」が「一番楽なやり方」になる



IDPが提供すべき機能

ゴールデンパス = 推奨される開発・デプロイの道筋。強制ではなく「舗装された道」を提供する。

主要なケイパビリティ

インフラストラクチャ

- セルフサービスプロビジョニング
- 環境管理
- セキュリティ自動化

開発者体験

- プロジェクトテンプレート
- ドキュメント自動生成
- 開発環境統一

デリバリー

- CI/CDパイプライン
- 観測可能性（Observability）
- フィーチャーフラグ

開発者が「本来の仕事」に集中できる環境を作る



プラットフォームチームのマインドセット

プラットフォームチーム = 「ツールを作るチーム」ではなく「プロダクトを提供するチーム」

従来のインフラチームとの違い

従来のインフラチーム	プラットフォームチーム
チケットで依頼を受ける	セルフサービスで使える
「作って終わり」	継続的に改善
使い方は各自で調べて	ドキュメント・サポート充実
強制的に使わせる	使いたくなるものを作る

名前を「プラットフォームチーム」に変えただけでは何も変わらない。開発者を**ユーザー**として扱い、彼らが「使いたい」と思えるプロダクトを提供する姿勢が必要。



アプリケーションは分散できた。データはどうする？

サービスを疎結合に分割し、IDPで各チームが自律的にデプロイできるようになりました。ただ、**データが中央集権のまま**だと、スキーマ変更のたびに全チームの調整が必要になり、結局そこがボトルネックになる。アプリケーションは独立したのにデータは共有——この矛盾を解消するのが、Ch.14のData Meshです。

Ch.14 Data Mesh

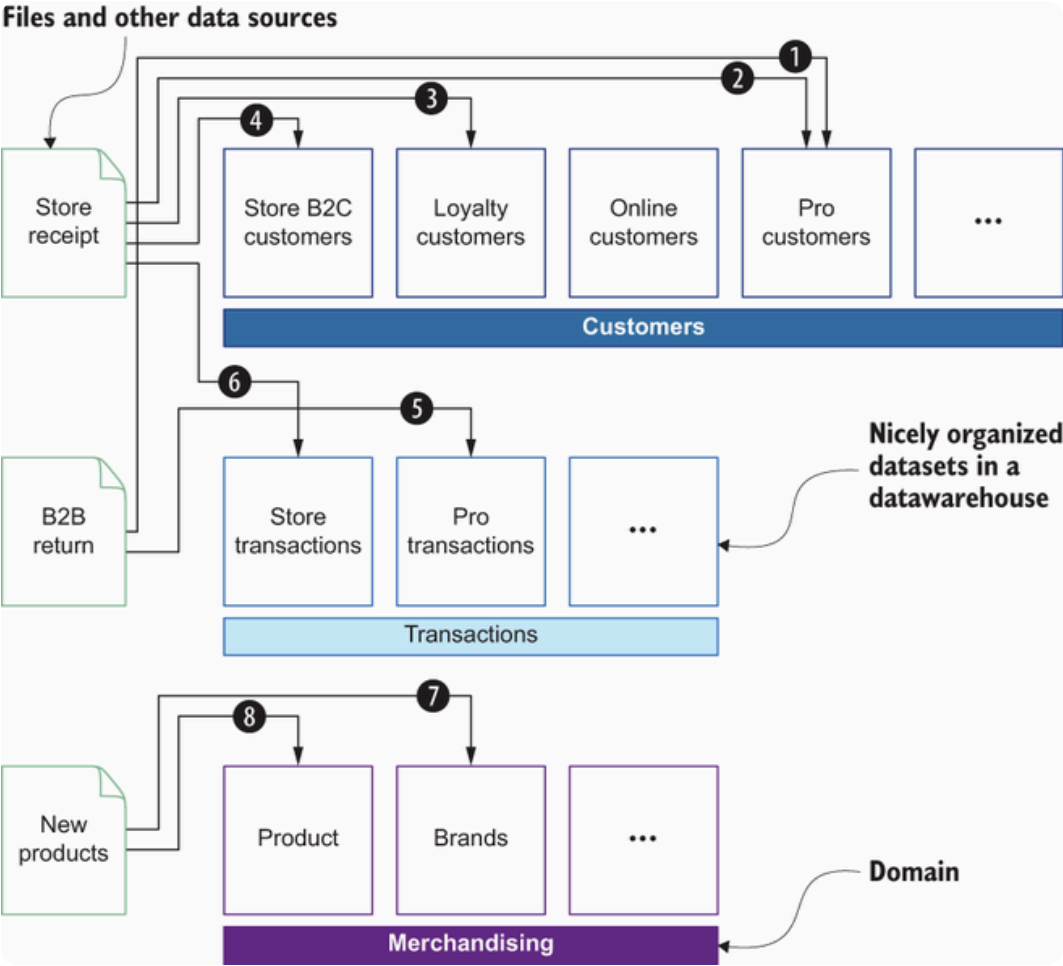


Figure 14.1 Data Mesh architecture より引用

Zhamak Dehghaniが提唱した分散データアーキテクチャ。マイクロサービスの思想をデータ基盤にも適用し、データの所有権をドメインチームに委譲する。

従来のデータ管理の問題

問題	結果
データチーム（中央）がすべて管理	ボトルネック化、待ち時間増加
ドメイン知識がない人がデータ整備	品質問題、誤解釈
「データの沼」にデータが溜まる	使われない、信頼されない

Data Meshの解決策

データを「プロダクト」として扱い、ドメインチームが責任を持つ。決済チームが決済データの品質・提供を担当し、在庫チームが在庫データの品質・提供を担当する。データを作る人がデータの品質に責任を持つ。

データを知っている人が、データを管理する



Ch.15 AMET（イネーブリングチーム）

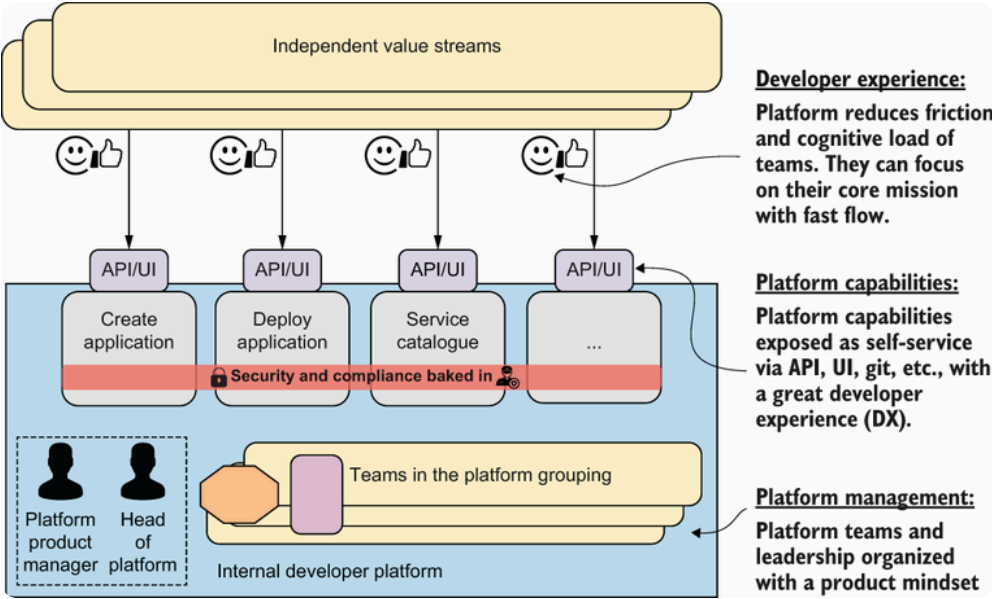


Figure 15.1 Architecture Modernization Enabling Team より引用

AMET = Architecture Modernization Enabling Team。モダナイゼーションを推進する専任のイネーブリングチーム。各チームが自力で変革を進められるよう支援し、やがて自分たちが不要になることを目指す。

なぜAMETが必要か

よくある問題	AMETがいると
「やり方が分からない」で止まる	ワークショップを設計・進行
「通常業務が忙しい」で後回し	専任が勢いを維持
「前のやり方に戻る」	継続的なコーチングで定着

AMETがやること・やらないこと

やること	やらないこと
ワークショップのファシリテーション	代わりにコードを書く
設計のレビューとフィードバック	決定を下す
学習コンテンツの提供	永久に伴走する

「魚を与える」ではなく「釣り方を教える」



Ch.16 戦略とロードマップ

失敗パターン: 「3年計画で全システムを刷新」 → 1年後に予算カット → 中途半端な状態で終了

成功パターン: Think Big, Start Small, Scale It

フェーズ	やること	具体例
Think Big	3年後のビジョンを描く	「リリースを週1回にする」
Start Small	1つのチームで成功体験	決済チームで3ヶ月POC
Scale It	成功パターンを横展開	他チームへ段階的に拡大

なぜ「小さく始める」が重要か

3年計画が失敗する理由は単純で、3年間も状況が変わらない前提が間違っている。予算カット、キーパーソン異動、市場変化—どれか1つは必ず起きる。だから**最初の3ヶ月で「やって良かった」を証明する**。小さな勝利が次の投資を引き出し、勝利の連鎖がモメンタムを生む。

「3年計画」は幻想。最初の3ヶ月の成果だけが、次の3ヶ月を保証する。



最後の、そして最も見落とされるピース

ロードマップもモメンタムもある。でも新しいアーキテクチャを設計・実装できる人がいなければ？ 外部コンサルが去った後に元に戻った経験、ありませんか。Ch.17は書籍の最終章として、変革を持続させる**学習文化**を扱います。

Ch.17 学習とスキルアップ

問題: 新しいアーキテクチャを設計しても、作れる人がいなければ絵に描いた餅

よくある失敗と成功のパターン

アプローチ	結果
✗ 外部コンサルに丸投げ	去った後に元に戻る
✗ 研修を1回やって終わり	2週間で忘れる
✓ 読書会を継続的に開催	知識が定着、実践に生きる
✓ 実際のプロジェクトで伴走	経験として身につく

具体的なアクション

期間	やること	効果
週1回30分	DDD/Team Topologies読書会	共通言語ができる
月1回3時間	イベントストーリーミング実践	手法が身につく
四半期ごと	振り返りと改善	組織に定着



「教わる」より「一緒にやる」が最強の学習

学習の具体的アプローチ

Community of Practice (CoP) = Etienne Wengerが提唱した実践共同体。同じ関心を持つ人々が自発的に集まり、知識を共有・深化させる場

効果的な学習形式

読書会

書籍を輪読し、実践への応用を議論

- 週1回、1章ずつ
- 実務との接点を意識

ハンズオン

実際に手を動かして体験

- イベントストーリーミング
- Wardley Mapping

振り返り

実践の経験を共有

- 成功・失敗の共有
- 改善アイデアの創出

必須スキル領域

DDD / Team Topologies / イベントストーリーミング / Wardley Mapping / 疎結合設計

学ぶことは、変わることの始まり



組織の学習成熟度レベル

学習成熟度 = 組織としてどれだけ学習が定着しているか。自組織が今どのレベルにいるかを把握することが出発点。

レベル	状態	特徴
Lv.1 無意識	学習の必要性を感じていない	「今のやり方で問題ない」
Lv.2 認識	必要性は感じているが行動なし	「勉強したいけど時間がない」
Lv.3 個人	個人レベルで学習	自費で本を買う、個人で勉強会参加
Lv.4 チーム	チームで学習	週1読書会、モブプログラミング
Lv.5 組織	組織として学習を支援	学習時間を業務時間に、予算確保

多くの組織はLv.2～3に留まっている。個人の熱意に依存している状態は、その人が辞めたら終わる。



書籍全体のメッセージ

17章を通じて書籍が一貫して伝えているのは、モダナイゼーションは技術プロジェクトではなく、ビジネス・組織・技術の3軸を同時に動かす変革だということ。どれか1つだけ変えても、残りの2つが足を引っ張る。

3つの統合

軸	内容
ビジネス	なぜ変えるのか、どこに投資するのか
組織	誰がどうやって変えるのか
技術	何をどう作るのか

技術だけ変えても元に戻る。組織だけ変えても形骸化する。
ビジネス・組織・技術を同時に動かせ。



どこから読むか

本音を言えば、Ch.1からCh.17まで順番に読んでほしい。著者も翻訳者も、その順番で読んだときに最も学びが深くなるよう設計している。それが本を作る側の願いです。

ただ、500ページ超の本を最初から読み通す時間がない、というのも現実。であれば、目的に応じて以下のルートから入ってください。

全体像を掴みたい

Ch.1 → Ch.2 → Ch.16

なぜ・どう進めるかの骨格

設計手法を学びたい

Ch.7 → Ch.9 → Ch.12

イベントストーリーミング→ドメイン→
疎結合

組織設計を学びたい

Ch.11 → Ch.10 → Ch.15

Team Topologies→ポートフォリオ
→AMET



本日のまとめ

書籍の全体像: Why → Discovery → Design → Organization → Technical → Execute の6パート・17章。技術だけでなく、ビジネス・組織・文化を同時に変革するソシオテクニカルなアプローチ。

翻訳者として一番伝えたいこと

モダナイゼーションの本質は、完成形を目指すことではない。**変化し続ける能力を組織に埋め込むこと**。技術を一新しても、変化に対応し続ける組織文化がなければ、3年後にはまたレガシーに戻る。「完成したシステム」は存在しない。「進化し続けるシステム」だけがある。その進化を支えるのは、小さなコミットメントを果たし続けた信頼の蓄積だ。

「500ページ読む時間がない」への回答

全17章を通して読まなくてもいい。Ch.7（イベントストーリーミング）、Ch.11（Team Topologies）、Ch.12（疎結合）——この3章だけでも実践に使える。ただし、Why（Ch.1-3）を飛ばすと「なぜやるか」の合意が取れず、結局つまづく。**Ch.1 → Ch.7 → Ch.11** の3章で約60ページ。これなら読める。

モダナイゼーションに「遅すぎる」はない。始めた瞬間から組織は変わり始める。



参考資料

- [Architecture Modernization](#) - Nick Tune, Jean-Georges Perrin (Manning, 2024)
- [アーキテクチャモダナイゼーション](#) - 日本語版、株式会社スリーシェイク訳
- [Team Topologies](#) - Matthew Skelton, Manuel Pais (IT Revolution, 2019)
- [Domain-Driven Design](#) - Eric Evans (Addison-Wesley, 2003)
- [EventStorming](#) - Alberto Brandolini
- [Building Microservices, 2nd Edition](#) - Sam Newman (O'Reilly, 2021)
- [Monolith to Microservices](#) - Sam Newman (O'Reilly, 2019)
- [Sooner Safer Happier](#) - Jon Smart (IT Revolution, 2020)
- [Data Mesh](#) - Zhamak Dehghani (O'Reilly, 2022)
- [Wardley Maps](#) - Simon Wardley
- [現代システムの三体問題](#) - Architecture Modernization 読書感想文
- [信頼と不信の哲学入門](#) - キャサリン・ホーリー (岩波新書, 2024)



The logo for 3 SHAKE, featuring the number '3' followed by a small icon of three overlapping colored triangles (yellow, green, blue) and the word 'SHAKE' in a bold, white, sans-serif font.

3 SHAKE

ありがとうございました

ご質問・ご相談はお気軽にお問い合わせください

@nwiizo | <https://3-shake.com>